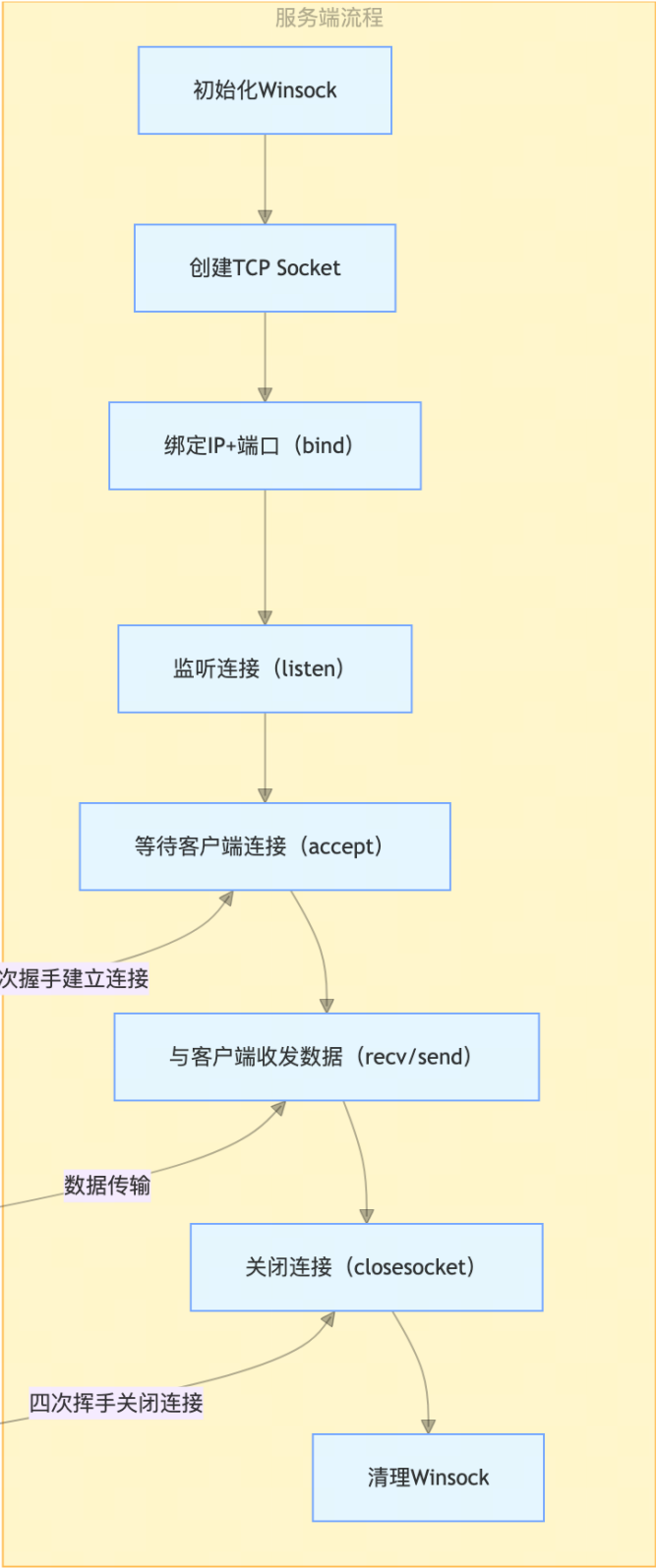
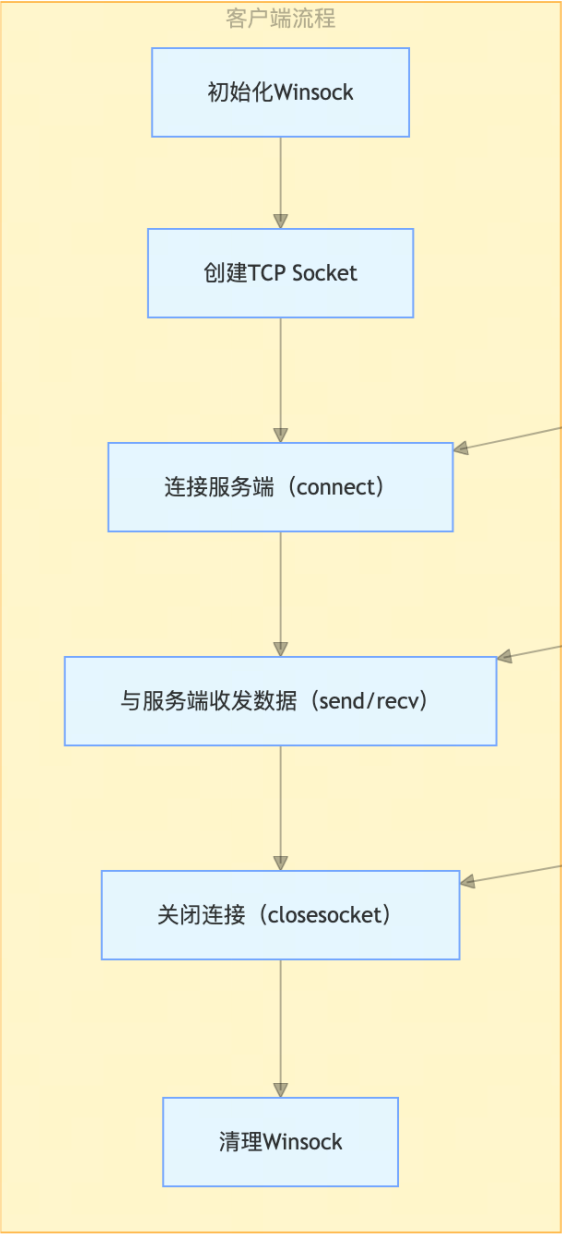


Socket

TCP



三次握手建立连接

数据传输

四次挥手关闭连接

版本号问题

🤔 考察了MAKEWORD (2, 2)、514这种写法

注意514是要转化为16进制0x0202，第一个02代表2.2中后面的2

然后这里版本号是要向下兼容的，如果请求2.2，但是系统只支持2.0，那么最终使用2.0

```
WORD sockVersion = 514;
WSADATA data;
if (WSAStartup(sockVersion, &data) != 0) return 0;
SOCKET sockLocal = socket(AF_INET, SOCK_DGRAM, 0);
if (sockLocal == INVALID_SOCKET) return 0;
```

#pragma加载动态链接库

```
#pragma comment (lib, "ws2_32.lib")
```

WSAStartup函数

Windows 平台专属，初始化 / 清理 Winsock 库（Linux 无需这一步）；

WSAStartup

```
1  int WINAPI WSAStartup(           // 成功返回0，否则为错误代码
2      WORD wVersionRequested,      // 调用者希望使用的最高版本
3      LPWSADATA lpWSADATA         // 可用socket的详细信息
4      );
```

🤔 版本位的解读：

- LOBYTE(wVersion)：取低8位 → 主版本号（比如2.2的低8位是2）；
- HIBYTE(wVersion)：取高8位 → 次版本号（比如2.2的高8位是2）；
- 2.0版本的 MAKEWORD(0, 2) → 低8位=2，高8位=0 → 十进制512（0x0200）。

如果遇到514，其实代表的是0x0202，但是分清版本号

如果初始化成功，返回值为0。

WSACleanup函数

WSACleanup 结束使用socket，释放Socket DLL资源

```
1  int WSAAPI WSACleanup();
```

如果操作成功，则返回值为零。否则，将返回值SOCKET_ERROR，并且可以通过调用WSAGetLastError检索特定的错误号。

socket函数

socket 创建一个socket，绑定到一个特定传输层服务

```
1  socket socket(  
2      int af,      //地址类型。AF_INET, AF_INET6等  
3      int type,    //服务类型。SOCK_STREAM、SOCK_DGRAM  
4      int protocol //协议。IPPROTO_TCP, IPPROTO_UDP, IPPROTO_ICMP  
5  );
```

- `af`：地址族 → `AF_INET` (IPv4)、`AF_INET6` (IPv6)；（不加6的就是ipv4，一般只会靠ipv4）
- `type`：Socket 类型 → `SOCK_STREAM` (TCP，字节流)、`SOCK_DGRAM` (UDP，数据报)
- `protocol`：协议 → `IPPROTO_TCP` (TCP)、`IPPROTO_UDP` (UDP)；（当这个参数传0的时候自动根据上一个参数选择协议）`IPPROTO_UDP` (17)
- 返回值：成功返回 Socket 句柄，失败返回 `INVALID_SOCKET`。

bind函数

这个函数是服务端专用的，用来绑定地址

Bind 将一个本地地址绑定到指定的socket

```
1  int bind(  
2      SOCKET s,      //socket描述符  
3      const struct sockaddr *addr, //地址。包括IP地址和端口号
```

```
4     int          namelen    //地址长度。通常为socketaddr结构长度
5 );
```

sockaddr结构体

代码块

```
1 // 2. 定义sockaddr_in结构体 (核心: 这里放INADDR_ANY)
2 sockaddr_in serverAddr;
3 serverAddr.sin_family = AF_INET;           // 地址族: IPv4
4 serverAddr.sin_port = htons(8080);         // 绑定端口 (转网络字节序), 这里固
    定端口为8080
5 serverAddr.sin_addr.s_addr = INADDR_ANY;   // 监听本机所有IP
6 serverAddr.sin_addr.s_addr = inet_addr("192.168.1.100"); //或者只监听这个ip地
    址
7
```

listen函数

这个函数用来建立监听状态，是进入监听状态，但是还没开始执行

listen 使socket进入监听的状态，监听远程连接是否到来

```
1 int listen(
2     SOCKET s,    //socket描述符
3     int backlog  //链接等待队列的最大长度
4 );
```

正确: 0

错误: SOCKET_ERROR

- `s`: 绑定后的监听 Socket;
- `backlog`: 监听队列长度 → 表示 “最多有多少个客户端处于「等待连接」状态” (比如 5);
- **关键**: 调用后 Socket 进入「被动监听状态」，但不会阻塞，直到调用 `accept()`。

accept函数

上面listen函数建立完监听状态之后，这个函数才真正开始执行监听，这里会阻塞

```

1 接受一个特定socket请求等待队列中的链接请求
2      SOCKET      s,
3      sockaddr    *addr,    //返回远程端地址
4      int          *addrlen //返回地址长度
5  );

```

`addrlen` 既要作为输入（告诉缓冲区大小），又要作为输出（返回实际长度），所以必须用 **指针参数**，否则调用者拿不到修改后的结果。

正确：**返回值**：新的 Socket 句柄 → 专门用于和该客户端通信，原监听 Socket 继续监听；

错误：INVALID_SOCKET

connect函数

这个是客户端发起连接的函数

connect 向一个特定的socket发出建联请求

```

1  int connect(
2      SOCKET s,                // 参数1: 客户端Socket句柄
3      const sockaddr* name,    // 参数2: 服务端的地址信息结构体
4      int namelen              // 参数3: 地址结构体的字节长度
5  );

```

正确：0

错误：SOCKET_ERROR（需用 `WSAGetLastError()` 查错误码）。

收发数据：`send()` / `recv()`

代码块

```

1  // 发送
2  int send(SOCKET s, const char* buf, int len, int flags);
3  // 接收
4  int recv(SOCKET s, char* buf, int len, int flags);

```

- `buf`：数据缓冲区；

- `len`：要发送 / 接收的字节数；
 - `flags`：通常传 0（阻塞模式）；
 - `send()` **返回值**：成功返回发送的字节数，失败返回 `SOCKET_ERROR`；
 - `recv()` **返回值**（TCP 核心）：
 - `>0`：成功接收的字节数；
 - `=0`：对方正常关闭连接（四次挥手完成）；
 - `<0`：接收失败（网络中断、超时等）。
-

closesocket函数

关闭 Socket 句柄，触发 **TCP 四次挥手**；

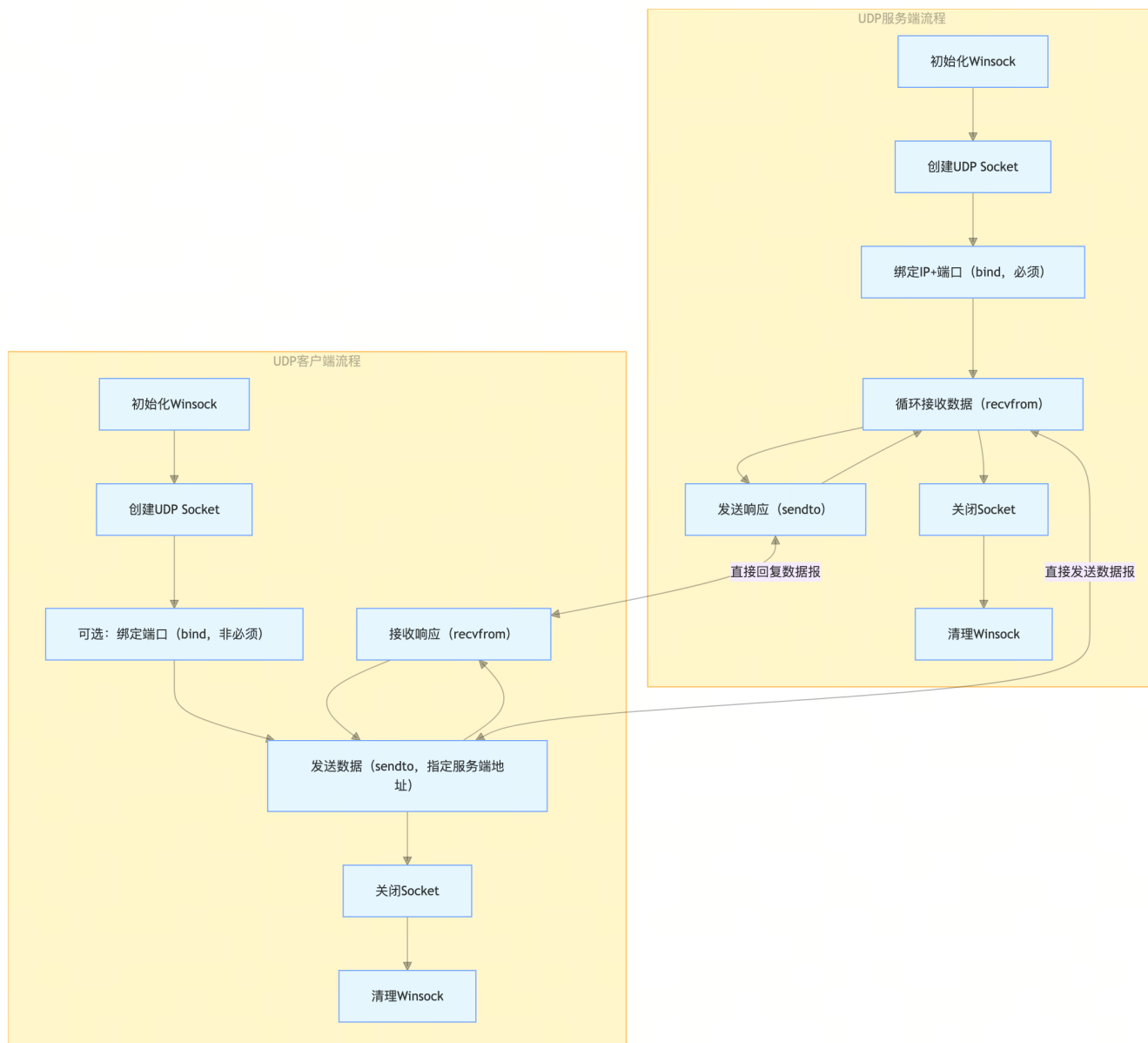
代码块

```
1  int closesocket(  
2      SOCKET s  
3  );
```

正确：0

错误：SOCKET_ERROR

UDP



UDP和TCP前期流程差不多，区别是客户端可以选择bind自己的地址

UDP 也可以调用 `connect()`，但作用不是“建立连接”，而是**固定目标地址**：

sendto函数

UDP 无连接，发送数据时必须指定「目标地址（服务端 / 客户端 IP + 端口）」，因此不用 `send()`，改用 `sendto()`：

代码块

```
1  int WSAAPI sendto(
2      SOCKET s,                // 参数1: UDP Socket 句柄
3      const char* buf,         // 参数2: 要发送的数据缓冲区
4      int len,                 // 参数3: 数据长度 (字节)
```



```

5         int flags,           // 参数4: 通常传 0 (阻塞模式)
6         const struct sockaddr* to, // 参数5: 目标地址结构体 (对方的IP+端口)
7         int tolen             // 参数6: 目标地址结构体长度
8     );

```

成功返回发送的字节数，失败返回 `SOCKET_ERROR`

recvfrom函数

代码块

```

1  int WSAAPI recvfrom(
2      SOCKET s,           // 参数1: UDP Socket 句柄
3      char* buf,          // 参数2: 接收数据的缓冲区
4      int len,            // 参数3: 缓冲区长度
5      int flags,          // 参数4: 通常传 0 (阻塞模式)
6      struct sockaddr* from, // 参数5: 输出参数, 存储发送方的地址信息
7      int* fromlen        // 参数6: 输入输出参数, 传入缓冲区长度, 返回实际地址
8      长度                // 参数6: 输入输出参数, 传入缓冲区长度, 返回实际地址
9  );

```

`from`：输出参数，接收后可知道“谁发的消息”（比如服务端收到后，用这个地址回复客户端）；

`fromlen`：必须先赋值为 `sizeof(struct sockaddr_in)`，再传入（否则会报错）；

关键注意： `recvfrom()` 是阻塞函数，直到收到数据才返回